

Computer Architecture

RTL Design & Verification of Cache Controller

with MSI Cache Coherence Protocol



Report Part 2

End term evaluation deliverables

Guide: Dr. Sujay Deb

Mentor: Ajinkya Ghawale

SUBMITTED BY

Group 18

Prerana Gaikwad – MT25172

Vanshika Sharma – MT25155

Shrishail Dolle – MT25147

Index

1. Memory Coherence	2
1.1 Problem with non-coherence cache	
1.2 What is memory-cache coherence Protocols	
1.3 Types of coherence protocols	
1.3.1 Write through coherence Protocol	
1.3.2 Write back coherence protocol	
2. MSI	4
2.1 What is MSI	
2.2 States in MSI	
2.3 FSM of MSI Protocol	
3. System Architecture	5
4. Design Choices	5
4.1 Memory block size	
4.2 Line size	
4.3 Associativity	
4.4 Replacement Policy	
4.5 Write operation policy	
5. Testplan	6
5.1 Validating state change of MSI Invalid to Modified	
5.2 Validating state change of MSI Invalid to Shared	
5.3 Validating state change of MSI Shared to Modified	
5.4 Validating state change of MSI Shared to Invalid	
5.5 Validating state change of MSI Modified to Shared	
5.6 Validating state change of MSI Modified to Invalid	
6. Testcase and Waveform	8
5.1 Validating state change of MSI Invalid to Modified	
5.2 Validating state change of MSI Invalid to Shared	
5.3 Validating state change of MSI Shared to Modified	
5.4 Validating state change of MSI Shared to Invalid	
5.5 Validating state change of MSI Modified to Shared	
7. Summery	23
8. Resources	24

Chapter-1: Memory Coherence

1.1. Problem with non-coherence cache

In a multi-core system , each core has its own cache copies to store the same data, when a single core writes into memory , only one cache copy gets updated and all processor cache copies become stale (outdated). [1]

1.2. What is memory Coherence Protocols

When multiple processors are writing into memory , after the write operation of a processor rest all the processor should get the correct or updated value. To implement this some set of instructions to be executed is called memory or cache coherence protocol.

To get updated value to each processor cache copy memory coherence protocols are used.[1]

1.3. Types of coherence protocols

1.3.1. Write Back coherence protocol

1. Write through Protocol

If multi level memories are used , once a core executes write operation then each and every memory gets updated with that value simultaneously.

Advantages:

- a. Main memory always consistent
- b. Simplest protocol

Disadvantage:

- a. Slower write operation : CPU have to wait till value is getting written into all memory blocks
- b. High memory traffic : every write operation generates traffic

1.3.2. Write back Protocol

If multi level memories are used then , once a core executes write operation then only the nearest memory block to cpu gets updated and corresponding address in other memory blocks becomes invalid. Write done into rest all memory blocks is done when that updated block is evicted from that memory Block.

Advantages:

- a. Faster write operation.
- b. Reduce memory traffic.

Disadvantage

- a. Complex protocol
- b. Possibility of data loss

Chapter-2: MSI

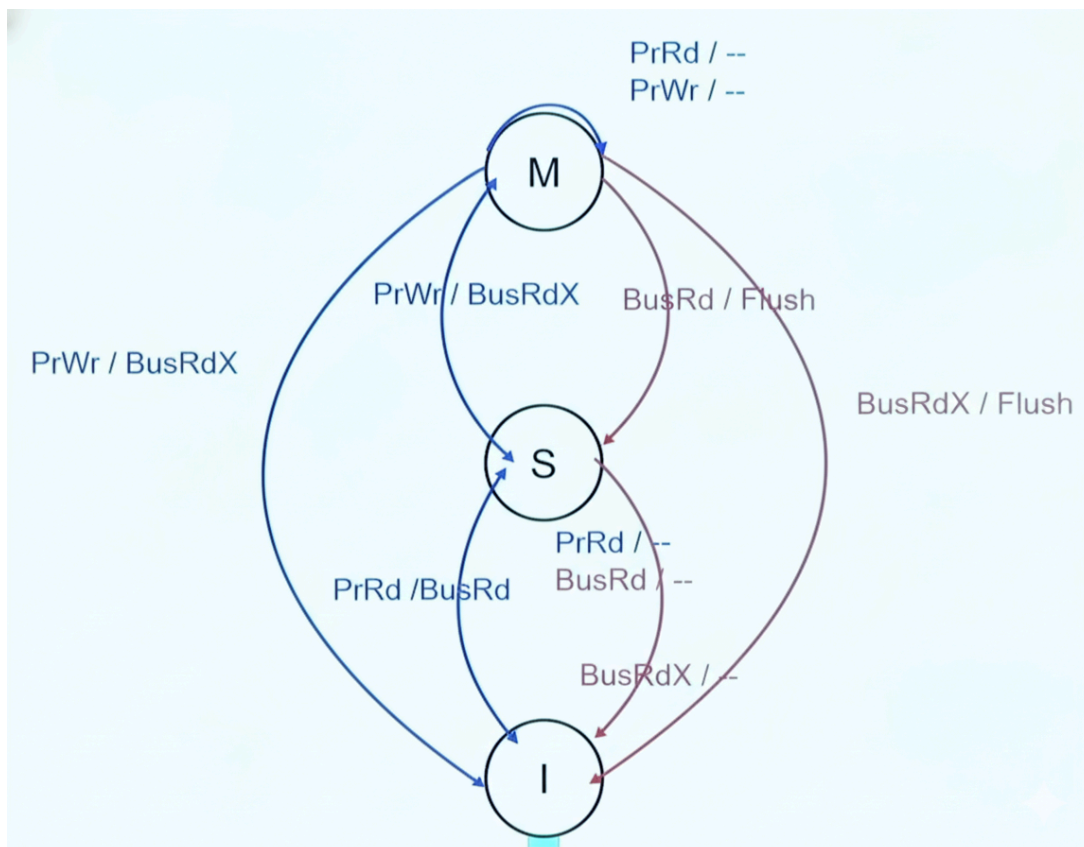
2.1. What is MSI

It is a fundamental cache coherence protocol used in multi processor systems to ensure all processors have a consistent view of main memory. It is an “invalid” protocol , when a single core writes into memory , the corresponding data block in another memory block becomes invalid. [3]

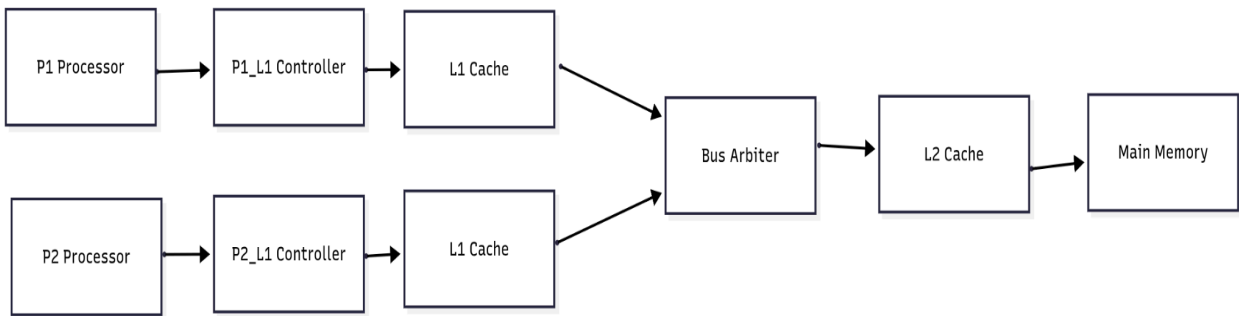
2.2. States in MSI protocol

1. Modify state
2. Shared state
3. Invalid state

2.3 FSM of MSI Protocol



Chapter-3: System Architecture



Chapter-4: Design Choices

4.1. Memory block size

L1 Cache : 1 kB

L2 Cache : 8 kB

Main memory size : 64 kB

4.2. Line size

L1 cache line bytes : 16 Bytes

L2 cache line bytes : 16 Bytes

Main memory line bytes : 16 Bytes

4.3. Associativity

L1 cache : 2 way set associative

L2 cache : 2 way set associative

4.4. Replacement Policy

We have used the Least Recently Used (LRU) block replacement policy because according to “Principle of Temporal Locality” there is a high probability of hitting memory again to that block or surrounding block. Hence replacing the least recent used block is preferred to reduce miss rates.

4.5 Write Operation Policy

P1_L1 , P2_L1 cache and main memory following write through policy

P1_L1 , P2_L1 cache and L2 cache following Write back policy

Chapter-5: Test Plan

5.1 Validating state change of MSI Invalid to Modified

- Initially all MSI states are Invalid.
- P1 processor sends write request to address 0x1010.
- After processing this write request the MSI Invalid state will be changed to MSI Modified state.

5.2 Validating state change of MSI Invalid to Shared

- Initially all MSI states are Invalid.
- P2 processor sends read request to address 0x1020.
- After processing this read request the MSI Invalid state will be changed to MSI Shared state.

5.3 Validating state change of MSI Shared to Modified

- Initially all MSI states are Invalid
- P1 processor sends read request to address 0x1040
- So the MSI state changes from Invalid to Shared.
- P1 processor sends write request to same address 0x1040
- Now the MSI state again changes from Shared to Modified.

5.4 Validating state change of MSI Shared to Invalid

- Initially all MSI states are Invalid.
- P2 processor sends read request to address 0x1060.
- MSI state of P2 changes from Invalid to Shared.
- P1 processor sends write request to address 0x1060
- Now MSI State of P2 address 0x1060 will change from Shared to Invalid

5.5 Validating state change of MSI Modified to Shared

- Initially all states of MSI are Invalid
- P1 processor sends write request to address 0x2000
- MSI state of address 0x2000 changes from Invalid to Modified.
- P2 processor sends read request to address on same address 0x2000
- After execution of the second request MSI state of P1 on address 0x2000 will change from Modified to Shared.

5.6 Validating state change of MSI Modified to Invalid

- Initially all states of MSI are Invalid.
- P1 processor sends write request to address 0x1080
- MSI state will change from Invalid to Modified.
- Now P2 will send a write request to address 0x1080.
- So MSI state for address 0x1080 will change from Modified to Invalid as P2 writes on the same address.

Chapter-6: Testcase and Waveform

6.1 Validating state change of MSI Invalid to Modified

Explanation of test case 1

Step 1 : Processor P1 sends write request to address 0x1010 along with data to write
0xEEEEEEFFFF11112222AAAABBBBCCCCDDDD

Step 2 : Initially all MSI state are invalid so P1_L1 cache will be invalid initially and dirty bit also be 0

Step 3 : Now bus is requested to enable BUS_RDX to invalidate all other data on same address

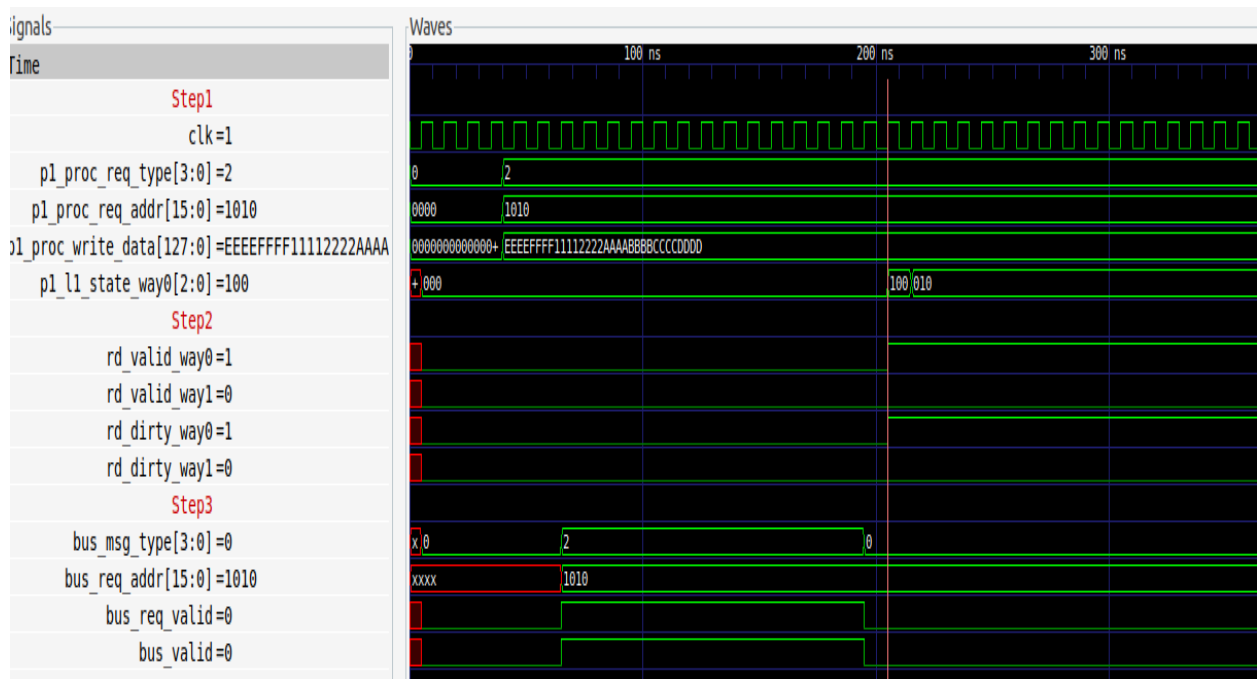
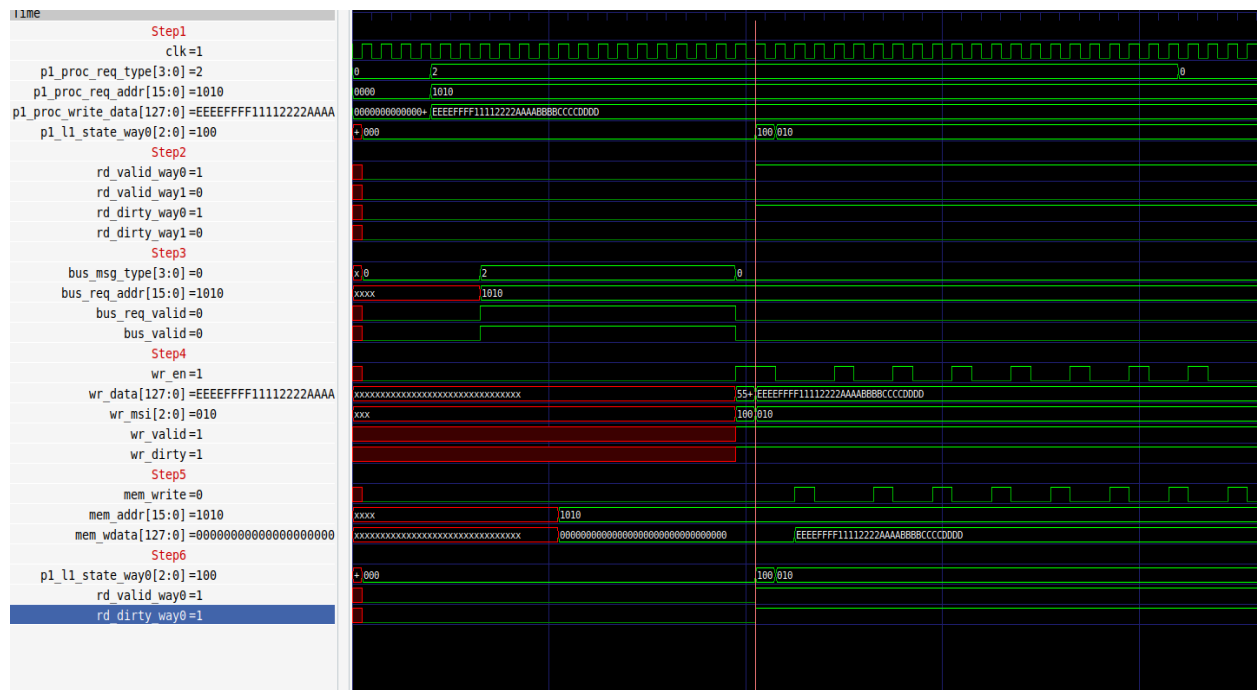
Step 4 : Now write operation to P1_L1 cache will held by enabling write mode on address
0x1010 and write data 0xEEEEEEFFFF11112222AAAABBBBCCCCDDDD

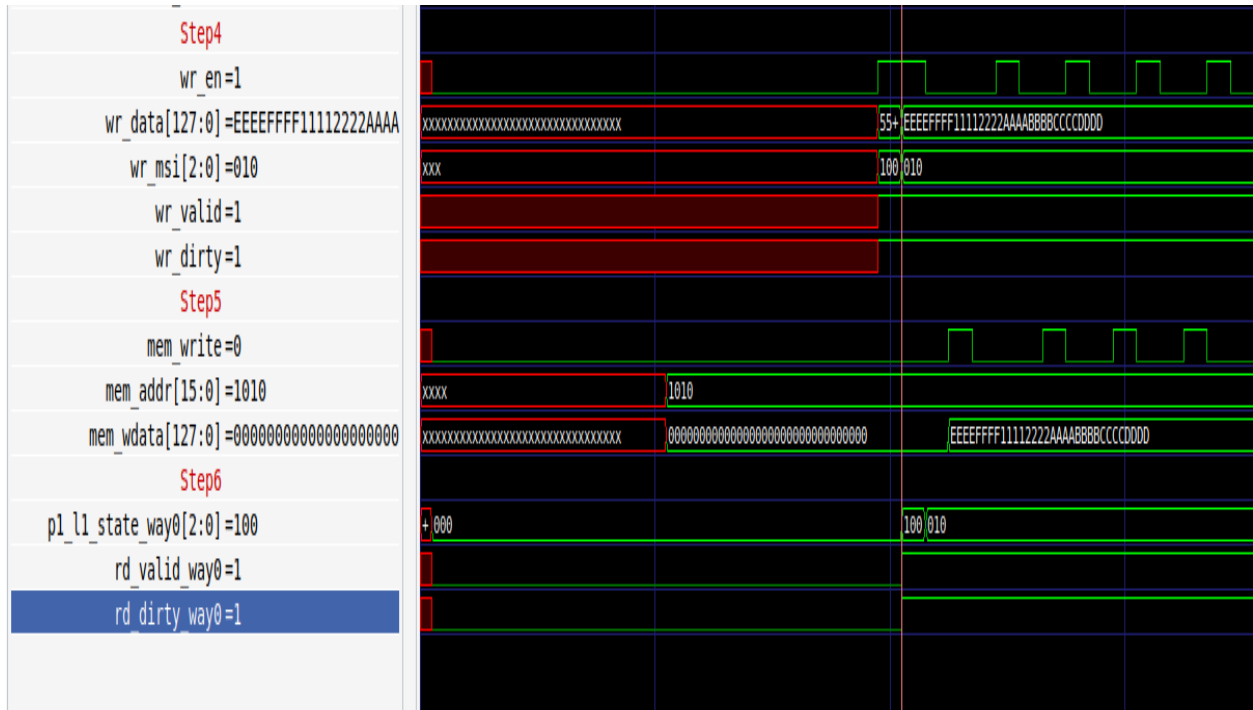
Step 5 : as memory and P1_L1 are following write through coherency protocol same data will be written into memory by enabling write mode on address 0x1010 along with data.

Step 6 : After execution of this request the state of P1_L1 state changes from Invalid to Modified before completion and after enabling write request the MSI state of P1_L1 changes from Invalid to Intermediate state ISD { Invalid-Shared Intermediate State }
So now dirty bit and valid bit become enabled showing write operation modified the data of P1_L1 cache.

Waveform for test case 1 :

P1 write request:





6.2 Validating state change of MSI Invalid to Shared

Explanation of test case 2 :

Step 1 : P2 processor sends read request to the address 0x1020 which address is not present in the P2_L1 cache initially and all MSI states are Invalid.

Step 2 : As address 0x1020 is not in P2_L1 hence it is P2_L1 cache miss.

Step 3 : As P2_L1 cache miss so the request is routed to L2 via bus enabling bus in read mode (RD)

Step 4 : As 0x1020 address is not present in L2 cache , cache miss takes place and request is routed to Main memory via memory controller.

Step 5 : Memory read done by enabling read mode of memory and corresponding data stored in memory will be forwarded to L2 cache via L2_Mem_controller

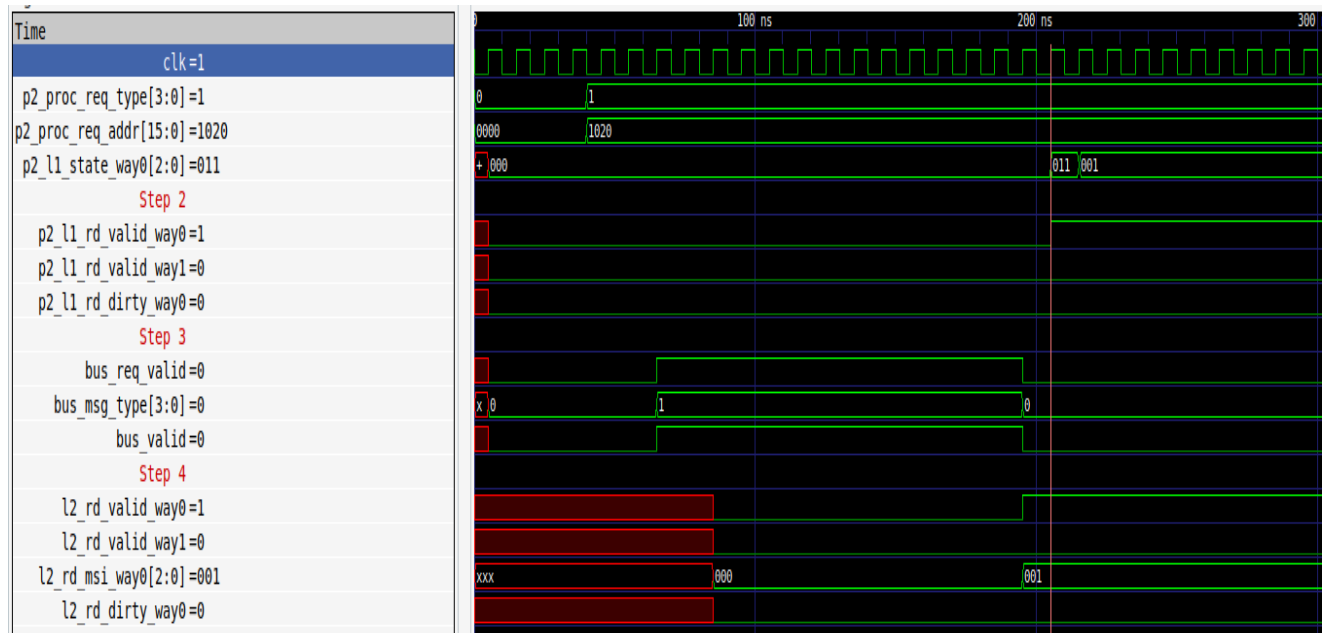
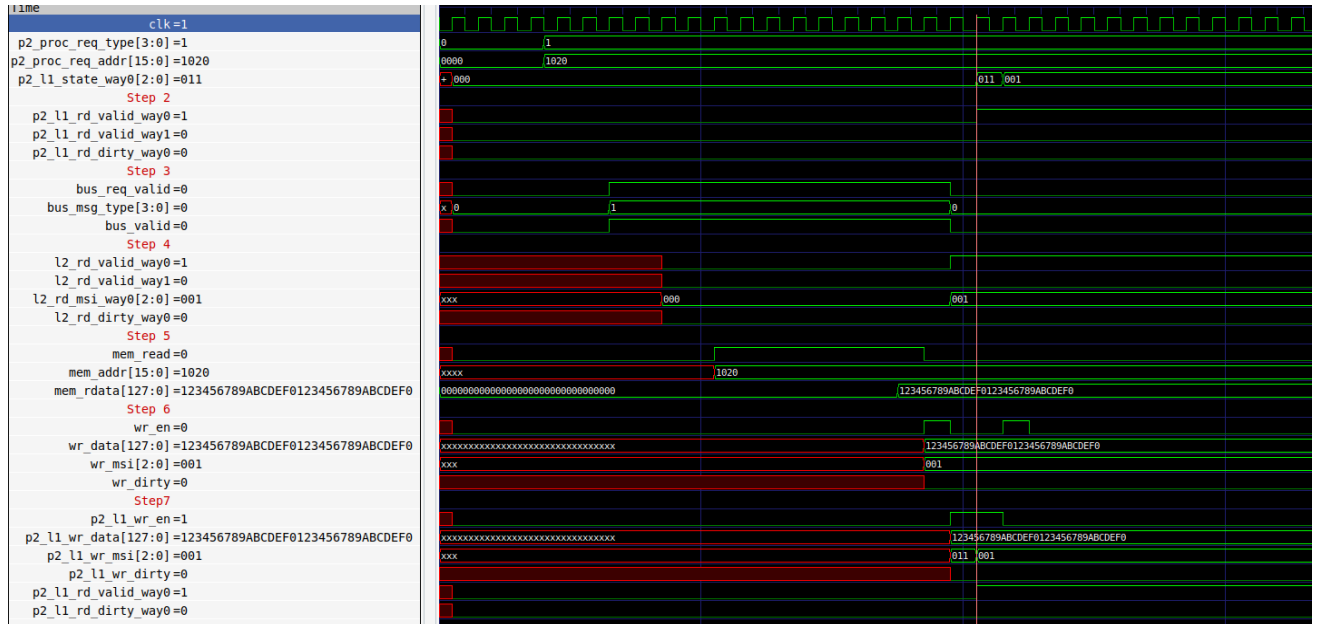
Step 6 : To obey coherency protocol the data read by memory is also written into L2 cache by enabling write mode but dirty bit is not set as write operation is done by L2_memory_controller to obey coherency not by processor. The data is not stale not modified by processor and it is valid data.

Step 7 : So finally the read requested data is written into P2_L1 cache also data is forwarded to the processor for processing . Data is written into P2_L1 cache by enabling cache in write mode.

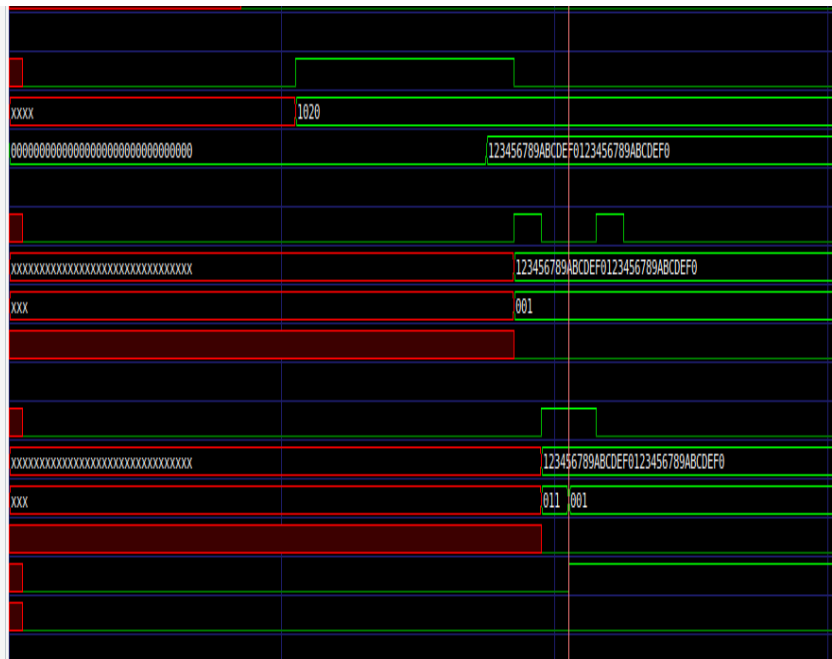
Step 8 : Finally MSI state of P2_L1 on address 0x1020 changes from Invalid -> ISD (Intermediate state for Invalid and Shared) -> Shared

Waveform for test case 2 :

P2 read req I to S



Step 5
mem_read=0
mem_addr[15:0]=1020
mem_rdata[127:0]=123456789ABCDEF0123456789ABCDEF0
Step 6
wr_en=0
wr_data[127:0]=123456789ABCDEF0123456789ABCDEF0
wr_msi[2:0]=001
wr_dirty=0
Step7
p2_l1_wr_en=1
p2_l1_wr_data[127:0]=123456789ABCDEF0123456789ABCDEF0
p2_l1_wr_msi[2:0]=001
p2_l1_wr_dirty=0
p2_l1_rd_valid_way0=1
p2_l1_rd_dirty_way0=0



6.3 Validating state change of MSI Shared to Modified

Explanation of test case 3 :

Here P1 requesting two operations first read then write

Step 1 : First request from P1 to read data from address 0x1040

Step 2 : As initially all MSI states are Invalid , the P1_L1 cache miss happens

Step 3 : As P1_L1 cache miss , the request is routed to L2 cache via bus , enabling bus in read mode

Step 4 : As it was initially invalid Address 0x1040 is not present in L2 cache , L2 cache also miss and request is routed to main memory via L2_Mem_controller.

Step 5 : Read operation executed in memory by enabling memory in read mode and address 0x1040.

Step 6 : To obey cache coherency , the data read done on main memory is written into L2 cache by enabling L2 cache in write mode. The write operation is done by P2_Mem_controller not by P1 processor so the dirty bit is not set and data is not stale.

Step 7 : The data is read by memory is also written into P1_L1 cache to maintain coherency and data forwarded to the processor for processing.

Step 8 : P1 processor sends write operation on same address 0x1040 along with data DEADBEEFCAFEBADEADBEEFCAFEBADE

Step 9 : The request is processed by writing the data into P1_L1 cache enabling the cache in write mode

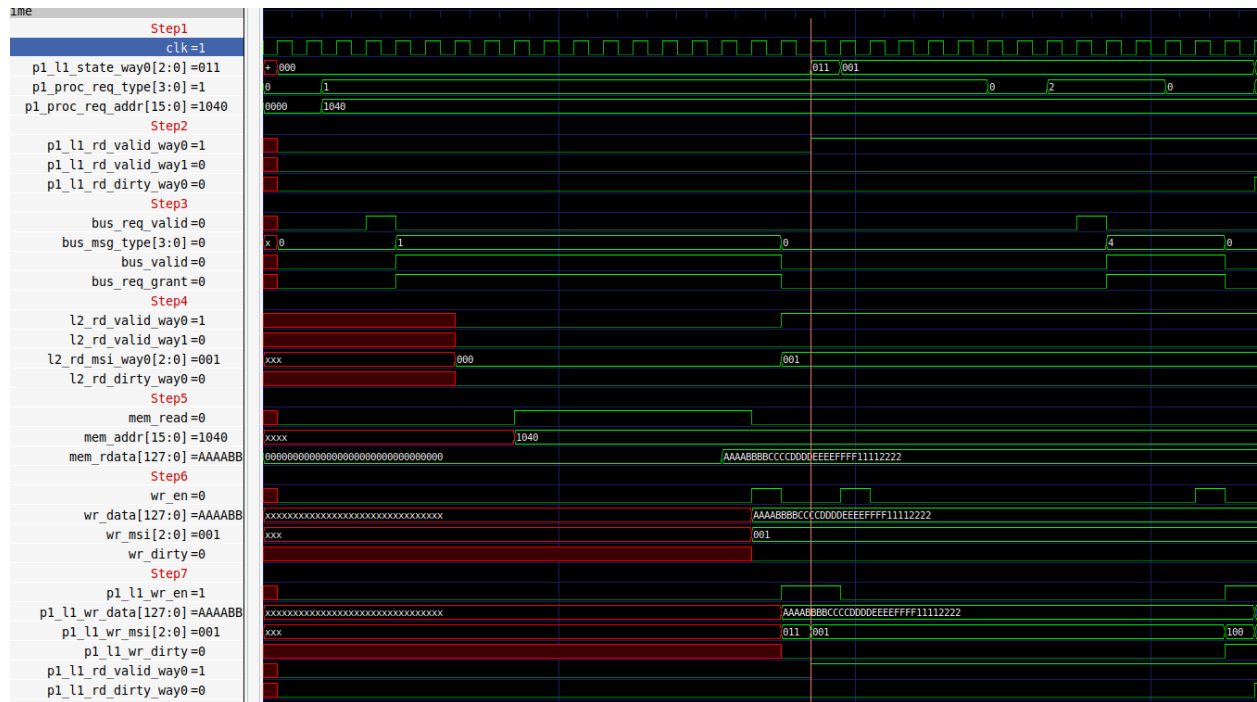
Step 10 : As P1_L1 cache and main memory are following write through protocol the same data will be written into memory by enabling write memory in mode on address 0x1040 along with data DEADBEEFCAFEBADEADBEEFCAFEBADE

Step 11 : now finally write operation is done and MSI state of P1_L1 on address 0x1040 is changed to Modified

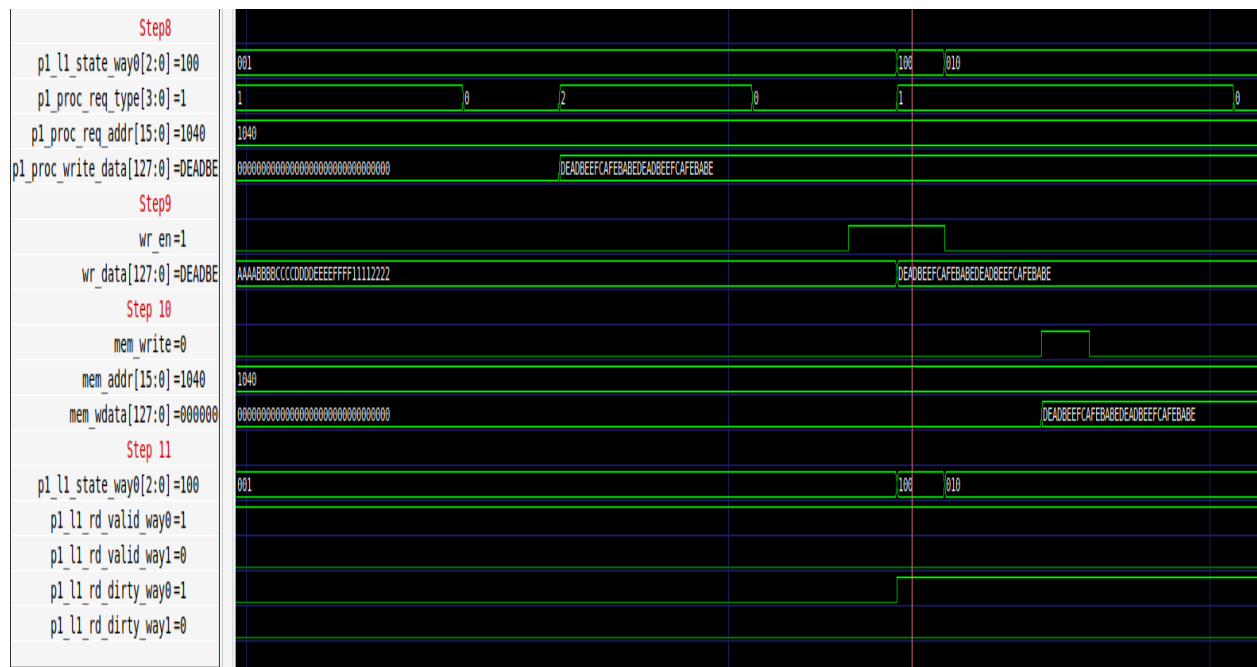
Step 12 : During entire process the MSI state of P1 has been changed from Invalid -> ISD (Intermediate State of Invalid and Shared) -> Shared -> IMD (Intermediate State of Shared and Modified) -> Modified

Waveform for test case 3 :

P1 read



P1 write



6.4 Validating state change of MSI Shared to Invalid

Explanation of test case 4 :

Here both processor P1 and P2 performing operation, P2 read operation and P1 write Operation

Step 1 : P2 processor send read request on address 0x1060 as initially all are invalid P2_L1 cache miss takes place.

Step 2 : As initially all are invalid so P2_L1 cache miss so valid bits are reset.

Step 3 : As P2_L1 miss so request is routed to cache L2 cache via bus , by enabling bus read in read mode

Step 4 : As initially all states are invalid so address 0x1060 is not present in L2 cache so it miss the L2 cache also. So the request is routed to memory via L2_Mem_Controller.

Step 5 : The read request is processed by main memory by enabling the memory in read mode Along with address 0x1060

Step 6 : To obey coherency , the data read done by memory is written into L2 cache by enabling L2 cache in write mode. Here dirty bit will not be set as data written by L2_Mem_controller not by processor , the data is not stale.

Step 7 : The same data read done on main memory is written into P2_L1 cache by enabling cache into write mode and sending data to processor for processing and here also dirty bit is not set as data is not modified by processor.

Step 8 : P1 sends write request on address 0x1060 along with data 99999999AAAAAAAA

Step 9 : This write request is routed through bus enabling bus in BUS_RDX mode to invalidate all other data values present on address 0x1060

Step 10 : The write request is processed by writing data on P1_L1 cache on address 0x1060 By enabling cache in write mode.

Step 11 : As we are following P1_L1 cache and main memory write through protocol same data is also written on main memory on address 0x1060.

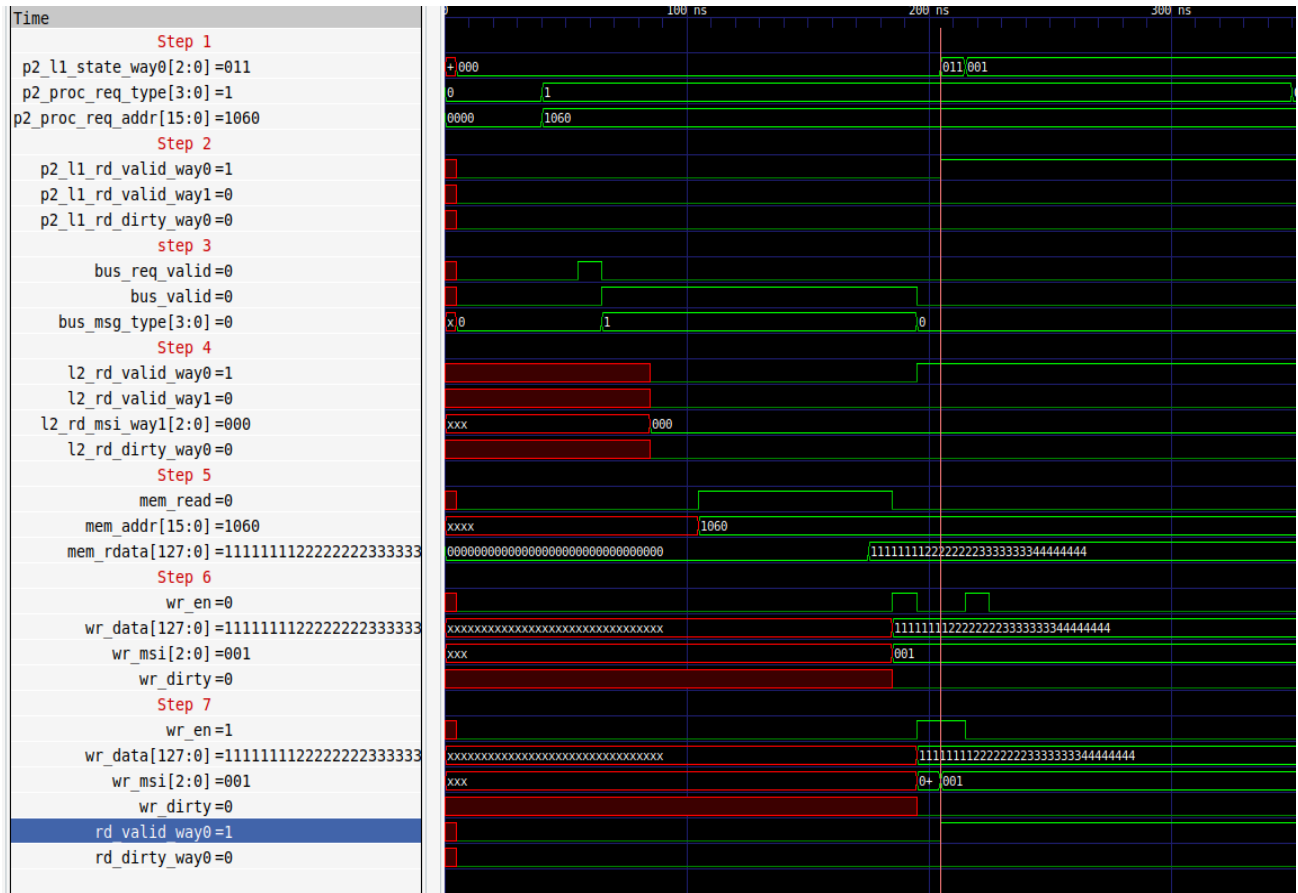
Step 12 : After the execution of both request the MSI states of respective processor changes with

P2 processor = Invalid -> ISD -> Shared -> ISD -> Invalid

P1 processor = Invalid -> IMD -> Modified

Waveform for test case 4 :

P2 read 1060



P1 Write 1060

Step	Signal Values
Step 8	p1_l1_state_way0[2:0]=100 p1_proc_req_type[3:0]=2 p1_proc_req_addr[15:0]=1060 p1_proc_write_data[127:0]=00000000000000000999
Step 9	bus_msg_type[3:0]=0 bus_req_valid=0 bus_valid=0
Step 10	p1_l1_rd_valid_way0=1 p1_l1_rd_valid_way1=0 p1_l1_rd_dirty_way0=1
Step 11	mem_write=0 mem_wdata[127:0]=000000000000000000000000+ mem_addr[15:0]=1060
Step 12	p1_l1_state_way0[2:0]=100 p2_l1_state_way0[2:0]=000

6.5 Validating state change of MSI Modified to Shared

Explanation of test case 5 :

Here both processors are performing operations.

P1 write operation , P2 read operation

Step 1 : P1 processor sends write request on address 0x2000 along with data
DEADBEEFCAFEBADEADBEEFCAFEBADE (all state initially in invalid)

Step 2 : this write request is drive by bus with mode BUS_RDX , to invalidate all other values
stored on address 0x2000.

Step 3 : Now the write operation is done on P1_L1 cache by making this cache in write mode,
Here the dirty bit will be set to notify that write operation is done by the processor.

Step 4 : As we are following P1_L1,main memory write through policy the data should also be
written into main memory by enabling memory in write mode

Step 5 : P2 processor sends read operation on same address 0x2000 , initially it is in invalid state.

Step 6 : As initially it is in Invalid state address 0x2000 is not present in P2_L1 cache hence
cache miss takes place and request is forwarded through bus

Step 7 : Now the request is routed via bus using bus in read mode.

Step 8 : As corresponding address's data is modified by processor P1 and processor P1's valid bit
is 1 , and its dirty bit is set ,P1 will forward the data to P2 to read.

Step 9 : The carry forwarded data from P1_L1 is written into P2-L1 by enabling P2_L1 in write
Mode.

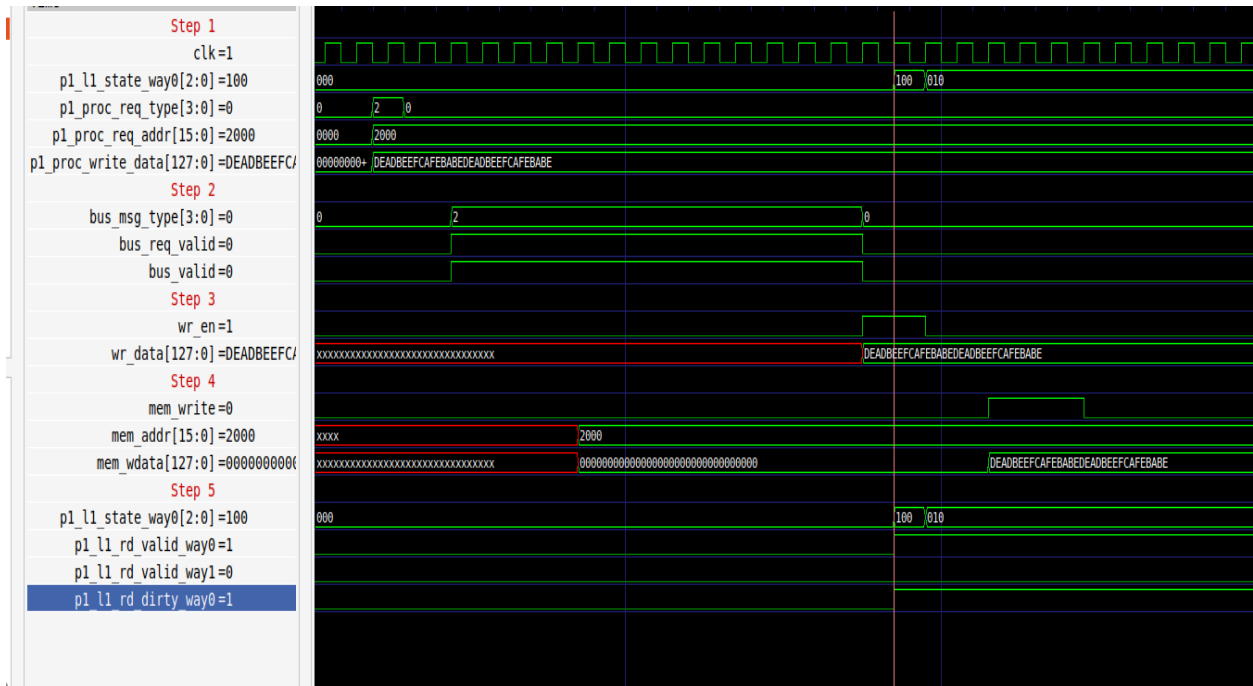
Step 10 : MSI states of both P1 and P2 are changing from

P1 = Invalid -> IMD -> Modify -> SMD -> Shared

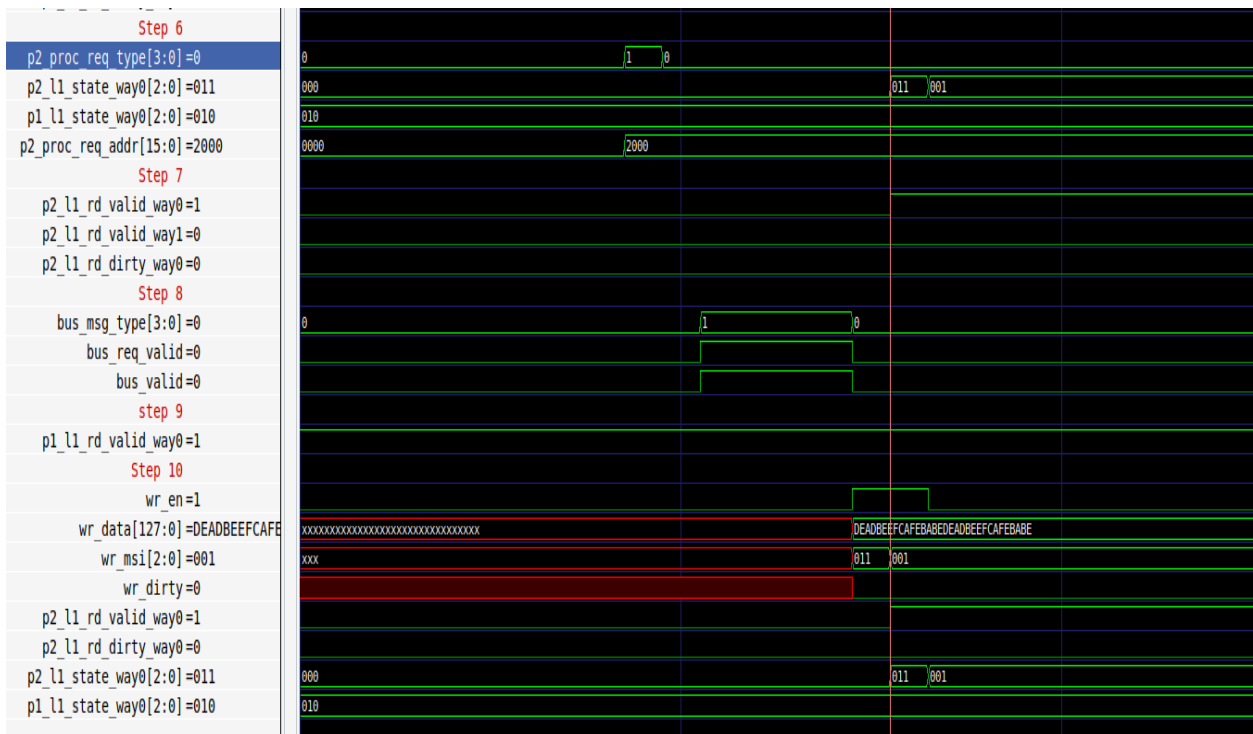
P2 = Invalid -> ISD -> Shared

Waveform for test case 5 :

P1 write



P2 Read



Chapter-7: Summary

- Implemented P1_L1 , P2_L1 , L2 cache , main memory specifications mentioned above.
- Implemented P1_L1_controller , P2_L1_controller , L2_Mem_controller as mentioned in architecture.
- Implemented bus arbiter to connect L1 and L2 caches.
- All modules are connected using msi_top.v
- Analysed all States of MSI protocol and also implemented using test cases and verified functionality.
- Implemented , analysed and verified intermediate states of MSI protocol.
- In testcase 1 : initially all states are invalid and write request done by P1 processor , after execution of this request , Invalid state changes to Modified
- Testcase 2 : Processor P2 makes a read request , initially all state are invalid , after processing this request ,MSI state changes from Invalid to Shared.
- Testcase 3 : Read request done by Processor P1 (which is initially Invalid state) , then make a write request so MSI state change of P1 is done from Invalid to Shared to Modified along with Intermediate states.
- Testcase 4 : P2 processor sends read request first (Initially Invalid) and then P1 processor make write request so MSI states of P2 changes Invalid to Shared to Invalid along with intermediate states.
- Testcase 5 : Processor P1 makes write request first (Initially Invalid) then P2 sends read request on same address so MSI state of P1 changes from Invalid to Modified to Shared state
- Testcase 6 : Processor P1 made a write request , on the same address P2 also made a write request , so MSI state changes of P1 Invalid to Modified to Invalid along with some intermediate state.

Chapter-8: Resources

- [1]. A primer on Memory Consistency and cache coherence [Morgan & Claypool publishers second edition] by Vijay Nagarajan , Daniel Sorin , Mark Hill , David Wood. resource
- [2]. ‘An Overview of On-Chip Cache Coherence Protocols’ III (A) [from IEEE Xplore] resources
- [3]. Lec 25 : 3 state : MSI protocol by Prof Hemangee K. Kapoor , Department of Computer Science and Engineering IIT Guwahati [Youtube] resources
- [4]. Git hub reference code for cache and memory implementation in verilog resources
- [5]. Used open AI for debugging code for controller and testbench.

Work Done By Group Members:

Implementation of MSI protocol.	Shrishail and Vanshika
Implementation of two processors and verification.	Prerana and Vanshika
Verification of results.	Shrishail and Prerana
Final changes.	All